

PRD: Audit Log under Account & Settings

1. Overview

This ticket introduces a new **Audit log** tab within the existing Account & Settings area of the platform. The surface gives Tenant Admins a single, self-service answer to the question *"who changed what, and when?"* — a question they currently cannot answer without contacting support or relying on informal channels.

At launch the log captures **course-settings changes only**. Critically, the surface is designed from day one to grow: subsequent releases will add new event types (enrolments, tenant settings, course content, etc.) and enable additional filter dimensions — all without changing the admin's mental model or navigation path.

The work tracked here (DES-318) is the **design pass** that feeds the engineering story DEV-4460. The design must cover the audit log tab, a filter bar with progressive-enablement semantics, a "Settings history" entry point on each course's Settings tab, and five explicit UI states (empty, populated, course-filtered, disabled-filter hover, and row hover).

2. Jobs To Be Done

Main Job

When something looks wrong on a course — a pass score that seems off, a compliance flag that was toggled, an enrolment rule that changed — the admin needs to quickly determine who changed it, when, and what the previous value was, so they can decide whether to revert, escalate, or simply confirm the change was intentional.

Job Map

Stage	What the admin does today	Where it breaks down
Define — Recognise that a setting seems wrong	Admin notices an unexpected behaviour or gets a learner complaint	No friction here — the trigger is external
Locate — Find evidence of the change	Asks colleagues on Slack, searches email, or files a support ticket	Major pain point. No self-service trail exists. Support must query DataDog manually.
Prepare — Gather context (who, what, when,	Waits for support to respond, often days	Major pain point. Context arrives late, is incomplete, and is not in a format the admin

before/after)		can share with stakeholders.
Confirm — Decide whether the change was intentional	Cross-references with the person who allegedly made the change	Relies on memory and trust; no source of truth.
Execute — Revert or accept the change	Manually re-edits the setting	No friction here if the admin knows the correct value.
Monitor — Watch for recurrence	No mechanism; relies on vigilance	Pain point. Without a log, recurrence goes unnoticed.
Resolve — Close the loop with stakeholders	Screenshots Slack threads or forwards support emails	Pain point. No exportable, shareable artefact.

Related Jobs

- **Compliance officer job:** "When preparing for an audit, I need a time-stamped, exportable record of every compliance-relevant change so I can demonstrate governance."
- **L&D lead job:** "When handing off a course to another admin, I want to see the full change history so I can brief them without forgetting anything."
- **Platform admin job:** "When diagnosing a system-wide issue, I want to see all changes across courses on a single screen so I can spot patterns."

Emotional & Social Dimensions

Dimension	Description
Functional	Answer "who changed what, when" in under 60 seconds without leaving the product.
Emotional	Feel in control and confident that nothing is changing behind their back. Move from anxiety ("I have no idea what happened") to certainty.
Social	Be perceived as a competent, well-informed admin by stakeholders and compliance reviewers. Be able to share evidence rather than anecdotes.

Hiring & firing: The admin is *firing* Slack threads, manual spreadsheets, and support tickets. They are *hiring* a persistent, always-up-to-date, self-service change log that lives where they already work.

3. Goals

1. **Self-service change investigation** — Admins can answer "who changed this setting, when, and what was it before?" without contacting support, within a single session.
2. **Zero-disruption extensibility** — The tab, table schema, and filter bar accommodate new event types and filter dimensions without layout shifts or mental-model changes.

3. **Trust through transparency** — Every recorded change shows a complete, field-level before/after diff, actor identity with role, and originating surface, so the admin never has to guess.
 4. **Design-system consistency** — The audit log tab, table, pagination, filter bar, and empty state reuse existing product patterns (same table pagination, same disabled-state colours, same tab overflow behaviour) so the feature feels native on day one.
 5. **Progressive enablement done right** — Disabled filters communicate future scope without creating confusion or false affordance. The "Coming soon" tooltip is specific enough to answer the obvious follow-up question ("what's coming?").
-

4. Job Stories

Investigation & Accountability

- **When** I notice a course's pass score has changed unexpectedly, **I want to** open the audit log and see who changed it and when, **so I can** decide whether to revert the change or confirm it was authorised.
- **When** I'm reviewing a compliance-flagged course, **I want to** see the full history of its compliance-status changes, **so I can** verify that the current state is correct and document the trail.
- **When** a colleague tells me they didn't change a setting, **I want to** look up the actor and surface for that change, **so I can** determine whether the change came from a different admin, a different surface, or an automated process.

Navigation & Entry Points

- **When** I'm already on a course's Settings tab and want to check its change history, **I want to** click "Settings history" and land in the audit log pre-filtered to that course, **so I can** see context without manual filtering.
- **When** I see the "Settings history" button shows a count badge of 12, **I want to** trust that number reflects all recorded changes for this course, **so I can** gauge how actively the course has been edited.

Filtering & Future Scope

- **When** I see greyed-out filters for Date range, Actor, Course, and Surface, **I want to** understand what they will do and when they'll be available, **so I can** plan my workflow and not feel confused by unusable controls.

- **When** I hover over a disabled filter and see a tooltip, **I want** the tooltip to tell me specifically what's coming, **so I can** set expectations with my team.

Empty & Edge States

- **When** I open the audit log tab for the first time and no changes have been recorded, **I want to** see a clear message explaining what will appear here, **so I can** understand the feature's purpose without guessing.

5. Requirements

Functional Requirements

FR1 — Audit Log Tab

ID	Requirement
FR1.1	Add an Audit log tab to Account & Settings, positioned next to a related tab or immediately after "Layout" to accommodate tab-bar overflow.
FR1.2	The tab is visible to Tenant Admins only . Non-admin roles must not see the tab.
FR1.3	The tab displays a paginated data table of all tracked changes for the tenant, sorted newest-first by default.
FR1.4	Pagination uses the existing product page-number pattern (page numbers, not infinite scroll).
FR1.5	The tab bar must handle overflow on smaller viewports (horizontal scroll or overflow menu).

FR2 — Table Columns

ID	Column	Notes
FR2.1	Course	Name of the course whose setting changed
FR2.2	Setting affected	Human-readable setting name
FR2.3	Previous value	The value before the change
FR2.4	New value	The value after the change
FR2.5	Actor	Name of the person (or "System" for automated changes)
FR2.6	Actor's role	Role at the time of the change

FR2.7	Surface	Where the change originated: "Settings tab", "Compliance configuration", or "System"
FR2.8	Timestamp	Date and time of the change, in the tenant's locale/timezone

FR3 — Row Behaviour

ID	Requirement
FR3.1	Each row links through to the relevant course's Settings tab. Row hover should provide a clear link-through affordance (e.g., pointer cursor, subtle highlight, or revealed action).
FR3.2	Rows reveal hover affordances on mouse-over. Do not show link/action icons in the default (non-hover) state to keep the table clean.

FR4 — Filter Bar

ID	Requirement
FR4.1	Setting filter is the only interactive filter at launch. It allows the admin to filter by specific setting name(s).
FR4.2	Date range, Actor, Course, and Surface filters are rendered in their final positions but in a disabled state using design-system disabled colours and <code>cursor: not-allowed</code> .
FR4.3	Hovering a disabled filter shows a tooltip : "Coming soon" — but the copy should be specific enough to avoid fuzzy context (e.g., "Date range filtering — coming in a future update").
FR4.4	Disabled filters must maintain their spatial footprint so the layout does not shift when they are enabled later.
FR4.5	Active filter selections are visually indicated (e.g., chip/tag pattern). A "Clear" or "Reset" mechanism is available when a filter is applied.

FR5 — Settings History Entry Point

ID	Requirement
FR5.1	A "Settings history" button (label TBD) is placed next to "Update Settings" on every course's Settings tab.
FR5.2	The button displays a count badge showing the number of recorded changes for that course. The badge updates in real time (not just on page load).
FR5.3	Clicking the button navigates to the Audit log tab with the Course filter pre-applied to the current course.
FR5.4	The button and badge are visible to Tenant Admins only . For non-admin users, the button is hidden entirely (not shown disabled).

FR6 — States to Design

ID	State	Requirements
FR6.1	Empty state	Shown when no changes have been recorded. Must communicate system status ("No changes recorded yet"), explain what will appear, and optionally guide the admin toward a learning cue. No misleading "loading" states.
FR6.2	Populated list	Standard paginated table view with all columns, filter bar, and row hover affordances.
FR6.3	Course-filtered view	Arrived via "Settings history" deep link. The Course filter is visibly applied. The admin can clear it to see all changes.
FR6.4	Disabled-filter hover	Tooltip appears on hover over any disabled filter. Cursor is <code>not-allowed</code> .
FR6.5	Row hover	Visual affordance indicating the row is clickable/linked.

FR7 — Recording Rules (Design Implications)

ID	Requirement
FR7.1	An entry is only written when a setting's value actually changes (no-op saves produce no entry).
FR7.2	When multiple settings are updated in one save, each changed setting produces its own row .
FR7.3	Settings tracked at launch: Pass score, Awarded jewels, Assessment attempts, Allow access after due date, Requires electronic signature, Enrolment visibility, Compliance status, Disable background playback, Course categories, Course certificate.
FR7.4	Surface values at launch: "Settings tab", "Compliance configuration", "System" (pending technical feasibility confirmation).

FR8 — Scope Exclusions (This Design Pass)

ID	Exclusion
FR8.1	CSV export — out of scope for this design pass.
FR8.2	Enabling Date range / Actor / Course / Surface filters — follow-up.
FR8.3	Manager and Subject Expert access — follow-up.
FR8.4	Event types beyond course-settings changes — follow-up.
FR8.5	Retention / deletion policy — deferred (records kept indefinitely).
FR8.6	Backfilling existing DataDog logs — out of scope (no clean before/after diff, no role/surface attribution).

6. Research & Best Practices

Synthesis

The audit log pattern is well-established in enterprise software. Martin Fowler's canonical guidance emphasises simplicity: "The glory of Audit Log is its simplicity" — an append-only log of changes works best when temporal querying is loosely coupled from day-to-day operations, and the simpler the structure, the more extensible it is when new event types are added later ([Martin Fowler — Audit Log Pattern](#)). This directly validates the ticket's architecture: a separate audit collection, one row per changed setting, and a schema that accommodates new event sources without modification.

Recording discipline: Sonar's audit logging guide reinforces that logs should capture deltas only (log when a value actually changes) and keep audit tables separate from business tables to preserve query performance ([Sonar — Audit Logging Best Practices](#)). The DEV-4460 acceptance criteria already encode this — no-op saves produce no entries, and the audit collection is separate from the operational database.

Display pattern: Before/after values should be displayed as side-by-side field-level comparisons, not entire object dumps, per the DEV Community research on audit log paradigms ([DEV Community — Audit Log Paradigms](#)). The Velt SaaS audit trail guide further specifies capturing user identity with role context and UTC timestamps with millisecond precision ([Velt — How to Add Audit Trail](#)). Both of these are reflected in the column requirements (actor + role, previous/new value columns, timestamp).

Table UX: Pencil & Paper's enterprise data table research recommends left-aligning text columns, right-aligning numeric values, using line divisions rather than zebra stripes (which create confusion when layered with hover, disabled, and selected states), revealing actions on row hover rather than cluttering the default view, and using sticky headers ([Pencil & Paper — Data Table UX Patterns](#)). These principles should govern the audit log table's visual design.

Filter bar: The same Pencil & Paper source on enterprise filtering advises getting the default state right so main use cases are reflected without configuration ([Pencil & Paper — Filter UX Patterns](#)). Arounda's SaaS filter examples recommend chip/tag filters for immediate visual feedback and auto-apply behaviour for admin workflows where speed matters ([Arounda — Filter UI Examples](#)). Eleken's guide notes that collapsible advanced filter panels with property→value selection work well for structured data like settings and event types ([Eleken — Filter UI Examples](#)). For the launch state, the Setting filter as a single interactive dropdown with chip display for active selections is the right level of complexity.

Progressive enablement (disabled filters): UXPin's progressive disclosure article distinguishes progressive *enabling* from progressive *disclosure* and warns that treating tooltips as the sole mechanism for critical information is a common accessibility failure — provide a persistent, discoverable trigger alongside hover interactions. Missing affordances cause "capability invisibility" ([UXPin — Progressive Disclosure](#)). Lollypop's guide reinforces that disabled controls with explanatory tooltips signal future availability without creating confusion, but the disabled state must be visually distinct without being invisible ([Lollypop — Progressive Disclosure in SaaS](#)).

Empty state: NN/g's research on empty states in complex applications defines three core principles: communicate system status explicitly ("No records to display" rather than blank space), provide learning

cues about what the feature does, and provide direct pathways for action. Crucially, they warn against misleading messages that claim "no records exist" while content loads — this damages trust ([NN/g — Designing Empty States](#)).

Pagination: Setproduct's pagination research confirms that traditional page-number pagination works best for data tables used in auditing and comparison tasks, and rows-per-page selectors (10/25/50) give users density control ([Setproduct — Pagination UI Design](#)). This aligns with the product's existing table pagination pattern.

LMS competitive landscape: A comparison of Docebo, Cornerstone, and TalentLMS audit capabilities reveals a key lesson: Cornerstone has the deepest audit log (20+ years of refinement, version control, multi-jurisdictional tracking) but its admin UI is "consistently described as cumbersome and complex." TalentLMS has a known limitation where course retakes overwrite history. The takeaway is that audit depth without usable UI undermines the feature ([Schoolx — LMS Comparison](#)). Compliance-focused LMS platforms require time-stamped grade modifications, detailed change logs tracking original values, and data export for compliance reviews as baseline expectations ([Pifini — LMS Tracking & Compliance](#)).

Tooltip copy — avoiding "fuzzy context": The AI UX Playground audit trail pattern defines expected capabilities: search, filter by date or action type, and export ([AI UX Playground — Audit Trail](#)). NN/g's design-pattern guidelines remind us that "Visibility of System Status" is a core usability heuristic — the disabled-filter tooltip must communicate *what* is coming, not just *that* something is coming ([NN/g — Design Pattern Guidelines](#)).

UX References

App	Flow / Screen	Mobbin URL	Pattern Description	Relevance
Cloudflare	Audit log table with filters	View	Enterprise admin audit log with filterable event table, actor column, timestamp, and action details	Direct pattern match — enterprise admin audit log with filter bar and paginated table
1Password	Activity log / audit trail	View	Security-oriented activity log showing who did what and when	Actor + action + timestamp pattern in a security-sensitive admin context
Gorgias	Activity/change log in admin	View	Admin activity log with event details and filtering	SaaS admin change log with structured event rows
Dropbox	Admin activity/audit view	View	Activity log showing user actions with timestamps	Mature product admin activity monitoring structure
Front	Admin audit/activity log	View	Change log with actor, action, and timestamp	Compact admin log layout with clear row

			columns	structure
Confluence	Page history / change log	View	Version history showing who changed what and when	Relevant as a "settings history" entry point pattern — per-entity change history
Retool	Settings change history with values	View	Change history showing previous/new values with actor and timestamp	Closest match to the before/after value display requirement
Discord	Server audit log	View	Audit log showing setting changes with old/new values and actor	Platform with many settings displaying granular change history
Better Stack	Activity/change log	View	Structured change log with timestamps and actors	Clean, modern audit log layout reference
Linear	Empty state in admin settings	View	Empty state with illustration and guidance text	Reference for the "no changes recorded yet" empty state
Shopify	Empty state in admin	View	Admin empty state with clear messaging and action pathway	How to communicate "nothing here yet" in an admin context
Workable	Empty state in settings/admin	View	Empty state with illustration and guidance	Alternative empty state pattern for admin features
Render	Filter bar with dropdown filters in admin	View	Admin filter bar with multiple filter dropdowns	Reference for filter bar layout with mixed active/inactive filters
Hotjar	Filter bar in admin panel	View	Admin filter bar with structured filter controls	Filter bar visual hierarchy in a SaaS admin panel
Linear	Filter bar with active filters	View	Active filter chips/tags in a filter bar	How active filters display as chips alongside inactive filter options
Reddit	Tab navigation with overflow	View	Horizontal scrolling tab bar with overflow handling	Tab-bar overflow pattern needed when Audit log tab is added
Ghost	Tab bar in admin settings	View	Settings tabs with multiple sections	Admin settings tab navigation pattern
HubSpot	Tab navigation in settings	View	Settings tab bar in a large admin panel	How a product with many settings tabs handles navigation

Built for Mars Lessons

Title	Type	builtformars.com URL	Lesson	Relevance
Building a POS Store	Case study	View	Square's empty customers page shows that totally empty screens with minimal affordances cause users to hunt for actions. The risk of a poor empty state isn't just user frustration — it's building the wrong thing based on misleading behavioural data (e.g., heavy search usage driven by poor table UX, not actual user preference).	Directly relevant to the audit log's empty state: the "no changes recorded yet" screen must clearly communicate what will appear and not leave admins guessing.
Grok — Onboarding AI Features	Case study	View	After a user completes their first action and the empty state transitions to a populated list, the onboarding shouldn't stop. No attempt to encourage further discovery means the feature flatlines after initial use.	Relevant to the transition from empty → populated audit log: once changes start appearing, consider guiding admins toward the filter bar and "Settings history" entry point.
Revolut's Progressive Onboarding	UX Bite	View	Revolut uses reactive tooltips after destructive actions to reassure users. Contextual, post-action tooltips are more effective than upfront onboarding.	Applicable to the "Coming soon" tooltip on disabled filters — reactive, contextual disclosure is more trustworthy than decorative labels. Also relevant to the "Settings history" entry point tooltip.
Fuzzy Context	UX Glossary	View	"A little context can create more ambiguity" — providing partial context can actually increase confusion rather than reduce it.	Directly relevant to the "Coming soon" tooltip copy: if the tooltip just says "Coming soon" without explaining what's coming, it may create more questions. Recommend specifying

				e.g., "Date range filtering — coming in a future update."
--	--	--	--	---

7. Plan of Action

Phase 1: Foundation — Tab, Table & Empty State

- ☐ **Tab placement & overflow:** Design the Audit log tab within Account & Settings. Position it after "Layout" or adjacent to a semantically related tab. Design the tab-bar overflow behaviour for smaller viewports (reference: [Reddit tab overflow](#), [HubSpot settings tabs](#)).
- ☐ **Table layout:** Design the 8-column data table (Course, Setting, Previous value, New value, Actor, Role, Surface, Timestamp). Left-align text, right-align any numeric values. Use line divisions (not zebra stripes). Sticky header. Reference: [Pencil & Paper data tables](#), [Cloudflare audit log](#), [Retool change history](#).
- ☐ **Row hover & link-through:** Define hover state (subtle row highlight + pointer cursor). Optionally reveal a "View course settings →" affordance on hover. Reference: [Pencil & Paper — reveal actions on hover](#).
- ☐ **Pagination:** Use the existing product page-number pagination component. Include rows-per-page selector if the existing pattern supports it.
- ☐ **Empty state:** Design the "No changes recorded yet" state with: (1) explicit status message, (2) learning cue explaining what will appear ("When course settings are changed, each change will be recorded here with full details"), (3) optional illustration consistent with product style. Reference: [NN/g empty states](#), [Linear empty state](#), [Shopify empty state](#), [BFM — Building a POS Store](#).
- ☐ **Admin-only visibility:** Confirm the tab is hidden for non-admin roles in all designed states.

Phase 2: Filter Bar — Active & Disabled States

- ☐ **Filter bar layout:** Design the full filter bar with all five filters (Setting, Date range, Actor, Course, Surface) in their final positions. Reference: [Render filter bar](#), [Hotjar filter bar](#).
- ☐ **Setting filter (active):** Design the dropdown/selection interaction for the Setting filter. Show active selections as chips with a clear/reset affordance. Reference: [Linear active filters](#), [Arounda chip filters](#).
- ☐ **Disabled filters:** Apply design-system disabled colours to Date range, Actor, Course, and Surface filters. Set cursor to `not-allowed`. Maintain spatial footprint identical to future enabled state. Reference: [UXPin progressive disclosure](#), [Lollypop progressive enabling](#).
- ☐ **"Coming soon" tooltip:** Design tooltip content that avoids fuzzy context. Recommended copy pattern: "[Filter name] filtering — coming in a future update" (e.g., "Date range filtering —

coming in a future update"). Reference: [BFM — Fuzzy Context](#), [BFM — Revolut's Progressive Onboarding](#).

Phase 3: Settings History Entry Point

- ☐ **Button placement:** Design the "Settings history" button next to "Update Settings" on the course Settings tab. Ensure visual hierarchy subordinates it to the primary "Update Settings" action.
- ☐ **Count badge:** Design the badge showing the number of recorded changes. Specify real-time update behaviour (badge count refreshes without full page reload). Specify badge states: no badge when count is 0, numeric badge for 1–99, "99+" for larger counts (or follow existing badge conventions).
- ☐ **Deep-link behaviour:** Specify that clicking the button navigates to Account & Settings → Audit log tab with the Course filter pre-applied and visibly shown as an active chip. The admin can clear the filter to see all changes.
- ☐ **Admin-only visibility:** The button and badge are completely hidden for non-admin roles (not disabled — hidden).

Phase 4: Interaction States & Edge Cases

- ☐ **Course-filtered view:** Design how the Audit log looks when arrived at via the "Settings history" deep link — the Course filter is visibly applied, other filters remain in their launch state (Setting active, rest disabled).
- ☐ **Responsive / overflow considerations:** Ensure the 8-column table handles narrow viewports gracefully (horizontal scroll, column priority hiding, or responsive breakpoints).
- ☐ **Long value truncation:** Define how long setting values (e.g., a list of categories) are truncated in the table and whether a detail view is available.
- ☐ **Transition from empty → populated:** Consider a subtle first-time-populated indicator or guidance (reference: [BFM — Grok onboarding](#)).

Phase 5: Design QA & Handoff

- ☐ **State inventory:** Verify all five required states are designed (empty, populated, course-filtered, disabled-filter hover, row hover).
- ☐ **Design-system alignment:** Audit all components against the design system (colours, spacing, typography, button styles, tooltip styles, badge styles).
- ☐ **Annotation & specs:** Annotate designs with interaction notes, tooltip copy, accessibility requirements (keyboard navigation for filters, ARIA labels for disabled filters, screen-reader announcement for tooltips), and Mixpanel event triggers.
- ☐ **Prototype update:** Update the Figma prototype (linked in the ticket) with final designs for engineering handoff.

- ☐ **Engineering walkthrough:** Present designs to the engineering team, review against DEV-4460 acceptance criteria, and resolve any open technical questions (e.g., "System" surface feasibility, real-time badge update mechanism).

8. Risks & Mitigations

Risk	Impact	Mitigation
Tab-bar overflow breaks on small viewports — Adding a 9th tab may cause layout issues on narrow screens or low resolutions.	Medium — Poor tab navigation undermines discoverability of the audit log.	Design explicit overflow behaviour (horizontal scroll with fade indicator or "More" dropdown). Reference Reddit and HubSpot tab patterns from the dossier. Test at the narrowest supported viewport.
"Coming soon" tooltips create more confusion than clarity — BFM's "Fuzzy Context" lesson warns that partial context increases ambiguity. A vague "Coming soon" label may prompt support tickets asking <i>what</i> is coming and <i>when</i> .	Medium — Undermines trust and increases support load.	Use specific tooltip copy: "[Filter name] filtering — coming in a future update." Avoid committing to dates. Validate copy with 2–3 admins before shipping.
8-column table is too wide for comfortable scanning — Especially on laptops (1280–1440px), eight columns may feel cramped or require horizontal scrolling.	Medium — Reduces readability and increases time to find information.	Prioritise column widths: Course and Setting get the most space; Role and Surface can be narrow. Consider combining Actor + Role into one column ("Jane Smith · Admin"). Test at 1280px.
Empty state persists too long for new tenants — If a tenant enables the feature but rarely changes course settings, the empty state may be the dominant experience, creating a perception that the feature is broken.	Low — Feature appears unused rather than valuable.	Craft an empty state that communicates the feature's purpose and trigger condition clearly. Consider whether a "Learn more" link to release notes or a help article would reduce confusion.
Real-time badge count is technically expensive — Keeping the "Settings history" badge count updated in real time (without page reload) may require WebSocket or polling infrastructure that doesn't exist.	Medium — Engineering complexity may delay the feature.	Discuss with engineering early. Acceptable fallback: update the badge on page load and after a save action on the same page, rather than true real-time push.
"System" surface value may not be technically feasible at launch — The ticket flags this as an open question. If	Low–Medium — Admins may see	Confirm technical feasibility before build. If infeasible, either omit "System" as a surface value at

automated processes can't be attributed, the Surface column may show blanks or misleading values.	unexplained changes with no actor/surface.	launch or display "Automated" with a tooltip explaining the limitation.
Backfill expectation gap — Admins may expect to see historical changes from before the feature launched. The absence of historical data may erode confidence in the log's completeness.	Medium — "Why does the log only start from July 2026?"	Address in the empty/first-use state: "Changes are recorded from [launch date] onward. Earlier changes are not available." Include in release notes.
Filter layout shift when filters are enabled later — Despite the requirement to maintain spatial footprint, enabling a filter may subtly change its visual weight, padding, or interaction area, causing a perceived shift.	Low — Minor visual inconsistency, but may confuse returning users.	Design the disabled state with the exact same dimensions, padding, and border as the enabled state. Use only colour/opacity to differentiate. QA both states side-by-side during design review.